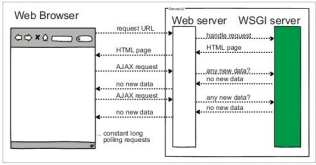

Figure 1: Long polling via AJAX

using Socket.IO, as it is really good and compatible with most of the mobile and desktop browsers used by people today. It is also based on Node.js and thus is very reliable for backend functionality too.

An introduction to Socket.IO

As discussed earlier, Socket.IO is a JavaScript library used to build real-time applications. We personally prefer it as it is nearly platform-independent -- the applications tend to function across all the browsers without hiccups. Another reason is that Node.js has a great, lightweight JavaScript library to perform back-end functions.

Socket.IO boasts of being 'the fastest and most reliable real-time engine'. Installing it is really simple.

So let us start by installing Socket.IO in your project. At present, there is just one way to install it -- through npm (node package manager). For that, you have to open up your terminal and type the following command (you need to have Node.js installed on your machine):

```
$ cd /path/to/project_dir
$ npm install socket.io
```

Getting started

Before we start on this, you are required to at least have some knowledge of the fundamentals of Node.js. Let us create a project named *test* for which we need the following:

- A computer/laptop with Windows, Linux or Mac installed with Node.js
- A text-editor like Notepad++, Emacs, Atom, Sublime, etc.

Create a project folder named *test* on your desktop (or wherever you prefer).

As the Socket.IO application is made using Node.js, the first and most important rule is to create a *package.json* file in the *test* folder with the following code:

```
{
        "name":"test_application",
        "version":"0.0.1",
        "private":"true",
        "dependencies":{
                "socket.io":"1.4.8",
                "express":"4.14.0"
        } }
```

Now, open your terminal/cmd and use the following commands:

```
$ cd /path/to/project_dir
$ npm install
```

After the process completes, browse the project folder *(test)* and you will find a *node_modules* directory. Open it and you will see the Socket.IO and Express directories there.

Now, make a new file named *test.js* and write the following code in it:

```
var express = require('express'),
    test = express(),
    server = require('http').createServer(test),
    io = require('socket.io').listen(server);
server.listen(3000);
test.get('/', function(req, res){
  res.sendFile(__dirname + '/welcome.html');
});


io.sockets.on('connection', function(socket){
            socket.on('sent', function(data){
                        io.sockets.emit('receive', data);
            });
});
```

Here, the Express server is initialised into the application and Socket.IO is initialised with the same server so that it is embedded into the app for further usage. The server is then started on Port 3000 on your local machine and can be accessed by using the URL *localhost:3000*.

When someone tries to access the URL while the server is active, a *get* request is passed on to the server; the request is then processed under the label 'req' and the response to this is labelled 'res' in terms of variables. The application then executes the function, which asks it to deploy the Web page named 'welcome.html' which resides in the same directory as *test.js*. Let's now create a file named *welcome.html* and put the following code in it:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Hurray!</title>
  </head>
  <body>
    <div id="content">
          <form id="name_form">
```